

CNN, RNN, LSTM

컨볼루션 신경망 (CNN: Convolution Neural Network)

1. 원본의 공간적인 특징을 평탄화(dense / fully connected network)하게 되면 특성을 찾지 못하게 된다.
2. 이를 해결하기 위해 Convolution(합성곱) 연산을 한다.
합성곱은 원본에 새로운 필터를 곱해서 새로운 특징을 만들어 내는것이다.
3. 필터 크기에 따라서 생성되는 가중치, 바이어스가 더 증가하게 된다
(Dense는 단 하나의 가중치/바이어스, CNN은 다수의 가중치/바이어스)
4. 합성곱(convolution 연산을 하게되면 정보의 양이 급증하게 된다.
5. 풀링을 통해 정보의 크기를 줄인다
풀링은 최대풀링/평균풀링 존재
6. 풀링의 역할은 정보요약 & 평행이동에 대한 불변
7. stride는 커널이 움직이는 거리, padding은 경계면 처리여부에 대한 것
8. 필터는 학습을 통해 최적화된다. (초기값은 랜덤)

CNN = 컨볼루션 신경망

"이미지의 지역적인 특성을 살리기 위해, 가중치를 공유하는 커널로 합성곱 연산을 하여 효율적으로 학습하는 신경망"

좀 더 풀어서 설명해봅시다



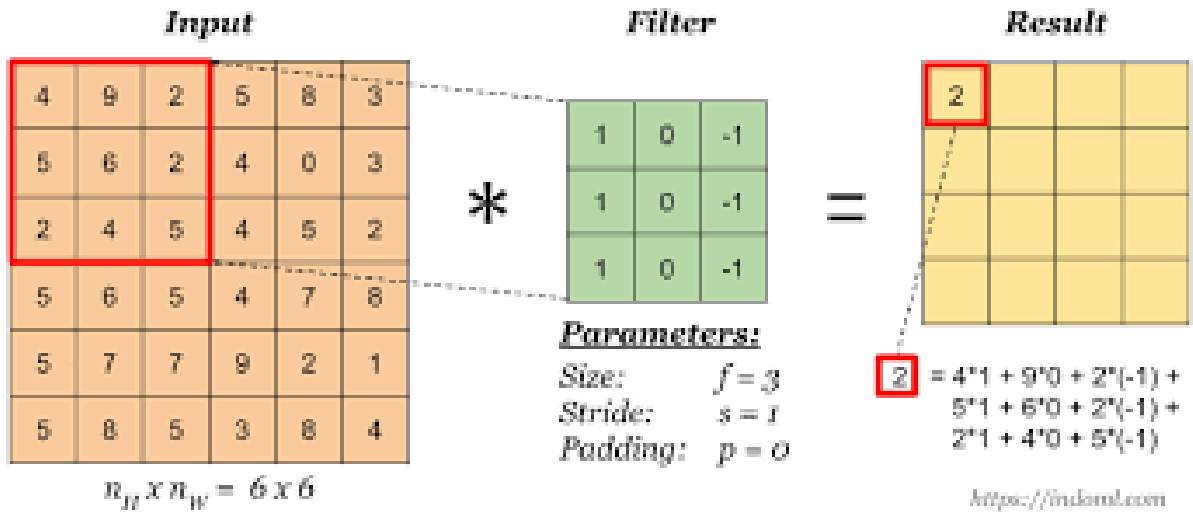
보통의 이미지는 2차원 행렬이기 때문에, 이 이미지를 학습하기 위해서 모델에 넣을 때는 1차원 행렬로 바꿔주는 **평탄화 작업**이 필요하다

다만 문제는, 이러한 평탄화 작업을 하면서 **이미지의 공간적인 특징이 소실된다는 것** (예를 들어, *코 아래에는 입이 있다* 같은 정보를 1차원 행렬에서는 찾기 힘들다)

그렇기 때문에 cnn에서는 **컨볼루션 연산**을 통해 이러한 문제점을 해결한다

컨벌루션 연산이란?

“이미지의 특징을 추출하기 위해 전체가 아닌 일부분을 확인한다”



특정 영역에 우리가 찾고자 하는 특징이 있는지 확인하기 위해 “필터(또는 커널)”를 옮겨가며 확인합니다.

필터는 적용하는 과정을 다음과 같습니다

- **겹치기:** 맵 위에 필터를 포개 놓습니다.
- **곱하기:** 겹쳐진 부분의 숫자끼리 서로 곱합니다. (이미지 숫자 $\times$ 필터 가중치)
- **더하기:** 곱해서 나온 숫자들을 모두 다 더해서 **단 하나의 숫자**로 만듭니다.
- **바이어스 더하기:** 나온 하나의 숫자에 바이어스를 더하고, 활성화 함수를 통과시킨다.



▼ 필터는 어떻게 생겼길래 특징 추출이 가능할까?

라는 궁금증을 해결하기 위해 모서리를 추출하는 **라플라시안 필터**를 봅시다

0	-1	0
-1	4	-1
0	-1	0

-1	-1	-1
-1	8	-1
-1	-1	-1

1	-2	1
-2	4	-2
1	-2	1

라플라시안 필터. 세 놈 다 라플라시안 필터이다

왜 이런 필터를 씌워서 적용시키면 모서리(엣지)를 추출해낼 수 있다는 것일까?

모서리를 잘 생각해보면 **“다른 부위보다 급격하게 어두워지는 경계선”** 정도로 정의할 수 있다

위 세 필터는 **모두 가중치의 합이 0이다**. 즉, 이미지의 평범한 흰색 배경에 적용시키면 모두 같은 값이 곱해지고 더해져 0이 된다.

반대로, 이미지의 밝기가 크게 변하는 모서리에 적용시키면 값이 0으로 상쇄되지 않고 큰 값이 튀어나오게 된다.

이렇듯 특정 가중치를 가진 필터들은 특정 특징을 추출해내는 성질을 가진다. 이 외에도 가로선을 추출하는 커널, 세로선을 추출하는 커널 등이 있을 것이라고 생각해볼 수 있다.

실제 cnn에서는 우리가 직접 커널의 가중치를 설정하는 것이 아닌, 모델이 학습하면서 조정하게 된다.



라플라시안 커널을 적용한 lenna image

그렇게 해서 나온 결과물을 **특징 맵(feature map)**이라고 합니다.

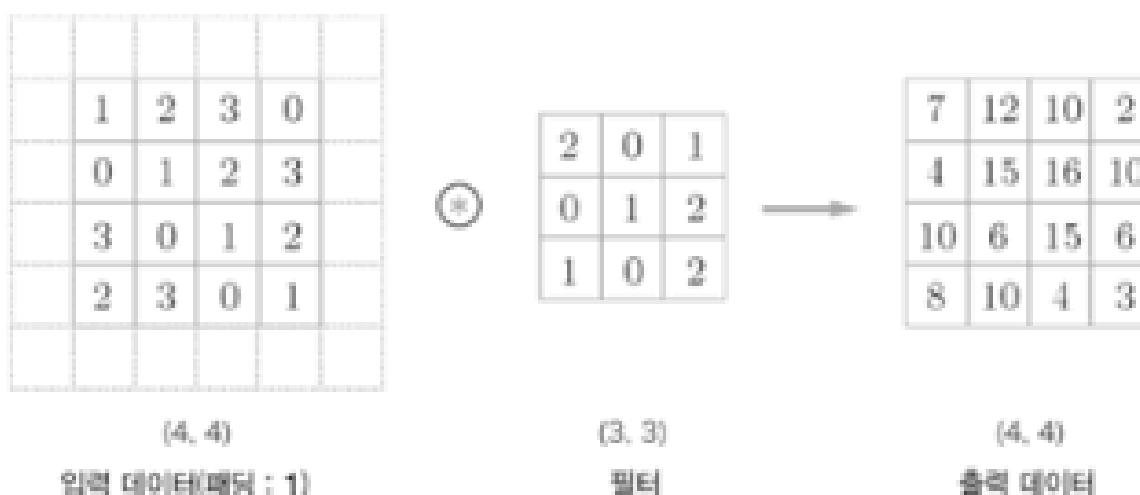
컨볼루션 연산은 이미지에서 특징을 추출하기 위해, 필터를 적용하여 특징이 담긴 특징 맵을 만드는 과정입니다.

그렇기에 다음과 같은 개념들을 알아야 할 필요가 있습니다.

- **stride**: 필터를 적용한 뒤 다음 필터를 얼마나 움직일 것인가
- **size**: 필터의 사이즈, 보통은 홀수로 정합니다(3X3, 5X5)
- **channel**: 컬러 이미지는 보통 $n * n * 3$ 형태로 되어있기 때문에, 필터도 각각의 컬러를 처리하는 필터 3개가 한 세트로 움직입니다
- **padding**: 이미지의 테두리를 처리하기 위한 추가 공간. 아래 서술
- **pooling**: 이미지의 정보를 압축하여 사이즈를 작게 만드는 기법. 아래 서술

Padding

패딩을 쓰는 이유: 맵의 가장자리 부분을 커널과 연산시키기 위해서



입력 데이터의 가장자리를 처리하기 위해서는, 위 그림처럼 여유공간이 필요할 때가 있다. 이러한 여유공간을 패딩이라고 한다

Same Padding

입력 데이터의 크기와 출력 데이터의 크기를 같게 만든다

가장 보편적인 패딩 전략으로, 합성곱 연산에서 크기가 줄어드는 것을 방지하기 위해 테두리를 두르는 전략

스트라이드가 1일때, 식은 다음과 같다 (P = 패딩, K = 커널 사이즈)

$$P = \frac{K - 1}{2}$$

Valid Padding

| Padding을 전혀 주지 않음 (P = 0)

사실 valid Padding을 계속하다보면 맵이 점점 작아지다가 정보가 소실되기 때문에 레이어가 많은 cnn에서는 잘 쓰이지 않는다

Pooling

중요한 정보만 남기고 쓸모없는 정보는 없앨 수 있다면 어떨까? 그럴 수 있다면 적은 리소스를 같은 결과를 낼 수 있으므로 안 할 이유가 없다. 그 중 하나가 바로 풀링이다

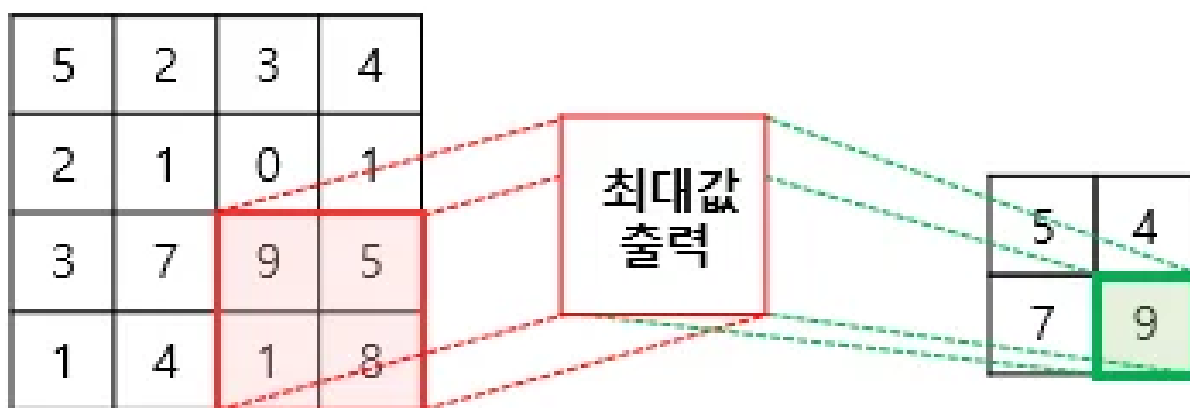
풀링이 뭘까? 특정 영역에서 특정 정보를 추출하는 기법이다. 보통 **맥스 풀링**과 **평균 풀링**이 있다



또한 풀링에서도 **패딩**이 있다! 대신 학습은 하지 않으므로 가중치는 딱히 없음
풀링 패딩의 목적은 커널의 패딩과 살짝 다른데,
크기 유지 목적이 아니고, 이미지 끝자락의 정보 보존의 목적이 큼

맥스 풀링 (Max Pooling)

| 특정 영역에서 가장 큰 값을 가져옴



보다시피 저렇게 최대값만 가져와서 저장한다. 크기가 줄어든다

평균 풀링 (Average Pooling)

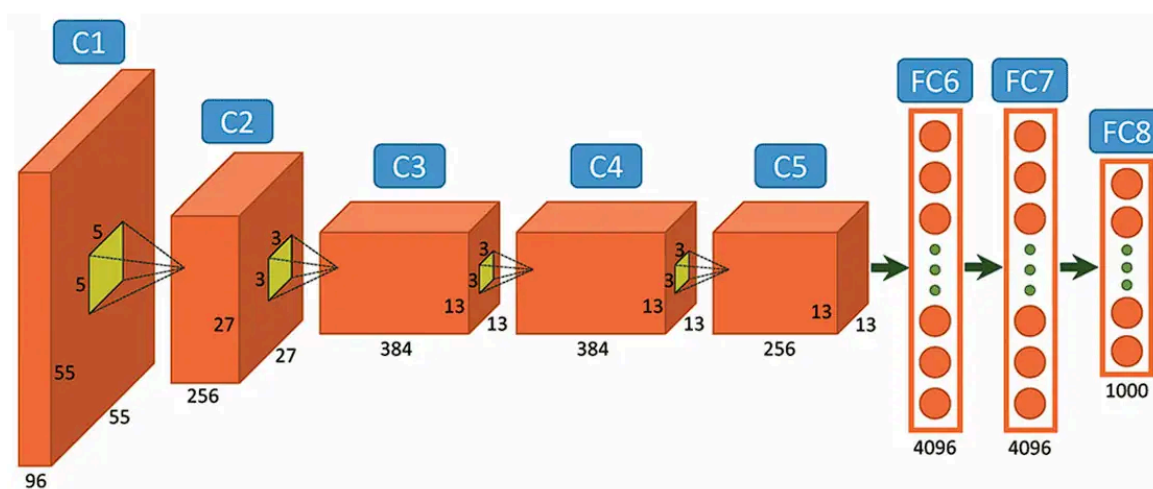
| 특정 영역의 평균을 가져와 출력

CNN

컨볼루션 신경망은 이러한 컨볼루션 연산을 계층적으로 쌓아 올리는 기법이다.

보통은 다음과 같은 과정을 따른다

1. 인풋 이미지를 준비
2. n개의 커널을 적용하고 활성화 함수를 통과시켜 이미지에서 특징을 추출(패딩, 스트라이드도 설정해줘야한다)
3. 풀링을 통하여 중요한 정보만 남기고 크기 축소(보통은 맥스 풀링을 활용함!)
4. **2~3번 과정을 반복한다**(반복학습). 즉, 특징을 추출한 특징 맵에서 다시 다른 커널을 통과시켜 특징을 찾고, 이 과정을 반복한다.
이 과정에서 특징 맵의 가로세로 크기는 작아지고, 커널의 개수(채널의 개수)는 점점 늘어나는 것이 일반적이다.
5. 마지막으로 가로 * 세로 * 채널 형태의 행렬을 1차원으로 평탄화시키고, 분류 또는 회귀를 한다



유명한 AlexNet의 구조

AlexNet의 구조를 보면... 갈수록 가로*세로 크기는 작아지고 채널의 개수는 많아지는 것을 알 수 있다

이를 통해, CNN에서는 레이어를 통과시킬수록 더 미세한 특징들을 더 많이 잡아낸다는 것을 알 수 있다

예를 들면 첫 번째 레이어에서는 단순한 선, 면, 점과 같은 특징을,
두 번째 레이어에서는 간단한 얼굴형, 눈, 코의 크기같은 특징을,
세 번째 레이어에서는 눈의 모양, 코, 입의 형태과 관계 등을 학습한다고 생각하면 된다

순환 신경망 (RNN: Recurrent Neural Network)

“표준 신경망의 구조를 순차 데이터를 잘 처리하게끔 변경한 신경망”

세상에는 순서를 가지는 데이터가 많습니다.

악보, 주식, 음성, 문장, 날씨 등등등...

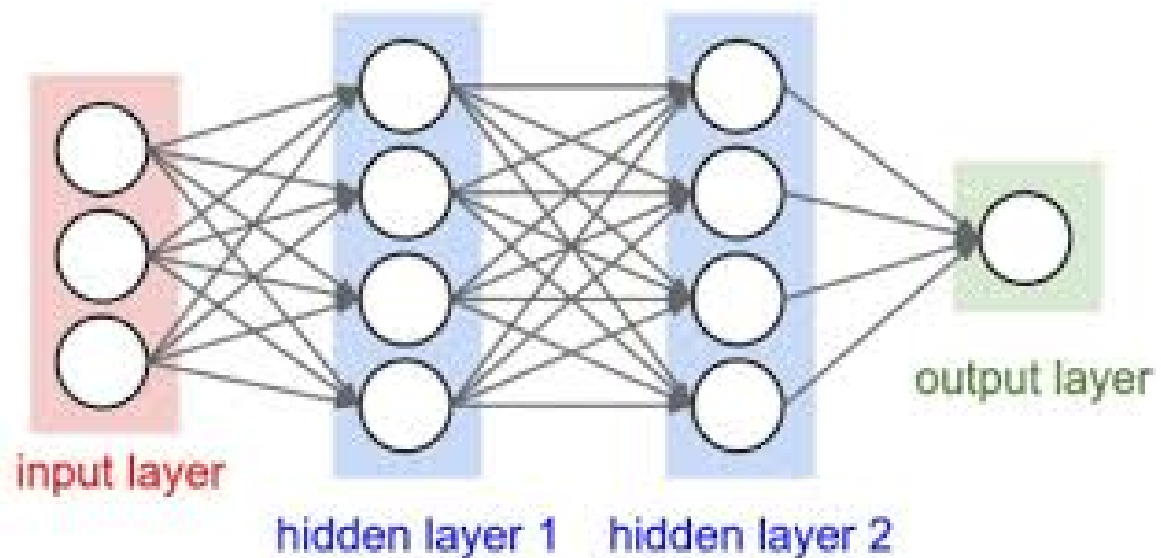
이러한 데이터를 처리하기 위해서는 현재 데이터의 특징만 보는 것이 아닌, **이전에 처리한 데이터와의 관계와 흐름**을 봐야합니다. 이 일이 바로 RNN이 하는 일입니다.

따라서 RNN은 다음과 같은 기능을 가져야 합니다

- 가변 길이의 입력 처리
- 장기 의존성 추적
- 순서에 대한 정보 유지
- 시퀀스 전체의 파라미터 공유

피드 포워드 신경망 (FFNN: Feed-Forward Neural Network)

“데이터가 입력층 ⇒ 은닉층 ⇒ 출력층까지 **오직 한 방향**으로만 흐르는 신경망”



우리가 배우는 RNN의 정반대의 개념입니다.

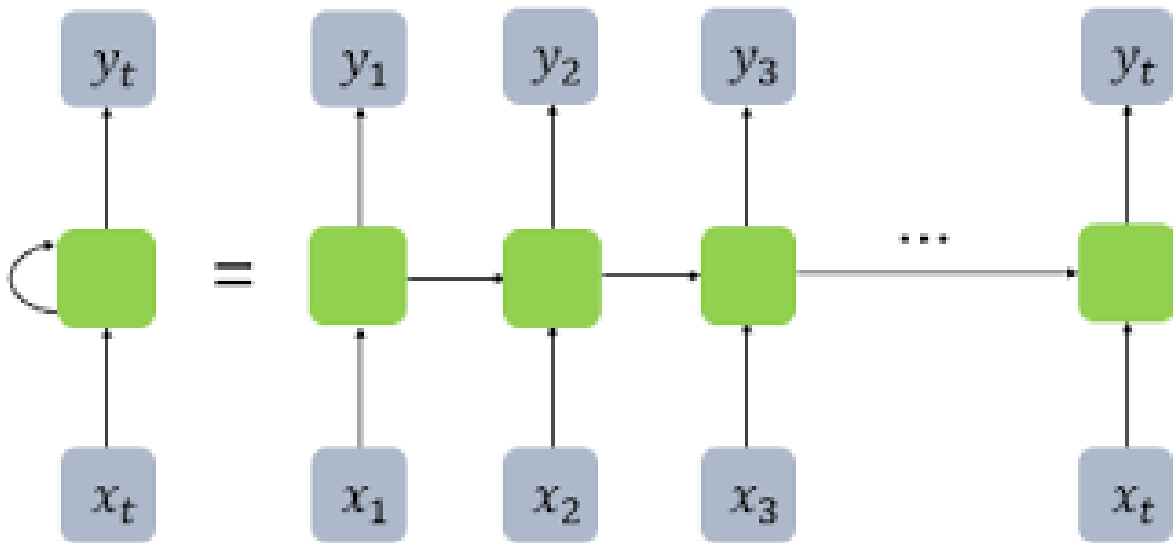
하지만 다음의 이유 때문에 시간에 따른 입력과 과거 데이터를 기억하는데에는 적합하지 않습니다.

- 입력 크기가 고정되어있음
 - 문장을 처리할 때 항상 같은 크기의 문자열만 처리 가능
- 위치가 다를 시 전혀 다른 데이터로 판단
 - "사과가 맛있다"와 "맛있는 사과가 있다"라는 문장을 학습할 때 두 사과를 완전히 다른 존재로 판단...
- 앞 뒤 데이터 문맥이 부재
 - "그는 **주먹**을 불끈 쥐었다" vs "그는 **주먹**밥을 먹었다" 와 같이 문맥에 따라 같은 단어도 다르게 판단해야하나, 이를 해결하지 못함

그래서 RNN은 어떻게 이러한 구조를 수정했을까요?

RNN의 경우

| RNN은 은닉층의 이전 출력을 다시 은닉층의 입력으로 넣는 구조를 가진다.



x_1 = 첫 번째 입력, x_2 = 두 번째 입력...이다.

y_1 = 은닉층의 첫 번째 출력, y_2 = 은닉층의 두 번째 출력... 이라고 이해하면 된다.

무슨 말인가 하면, 입력값이 다음과 같다고 생각해보자

입력값: ["나는", "사과를", "먹는다"]

만일 DNN구조였다면 각각의 3단어에 대한 3개의 가중치를 만든다. 이런 경우 단어가 길어지면 감당이 안되고, 단어 길이도 일정해야한다.

하지만 RNN에서는 다음과 같이 처리가 된다

1. 첫 번째 단어 "나는"을 처리해서 상태 h_1 을 만들고, 이 상태로 출력 y_1 을 낸다
2. 두 번째 단어 "사과를"과 h_1 을 같이 처리하여 h_2 를 만들고, 이 상태로 출력 y_2 을 낸다
3. 세 번째 단어 "먹는다"와 h_2 를 같이 처리하여 h_3 를 만들고, 이 상태로 출력 y_3 을 낸다

즉, **이전 데이터의 은닉상태(처리결과)를 현재 데이터에 적용시키는 구조**를 통해 이전 단계의 맥락을 기억하고, 순차 데이터를 처리할 수 있게 되는 것이다.

좀 더 깊게 들어가 보자. 데이터가 들어왔을 때 정확히 어떻게 처리하는 것일까?

기본적인 방식은 **모든 입력에 대해 총 3개의 가중치만 사용하기**이다

그 3가지 가중치는 다음과 같다.

- W_{xh} = 현재 입력에 대한 가중치
- W_{hh} = 이전 상태에 대한 가중치
- W_{hy} = 현재 은닉상태에서 출력을 생성하는데 사용하는 가중치 행렬

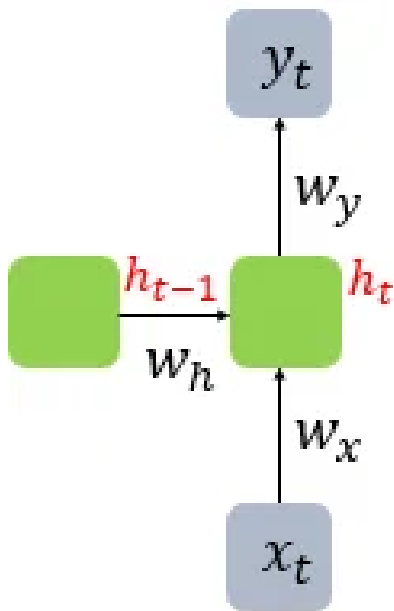
즉, 현재 상태를 구하기 위해 현재 입력과 이전 상태에 각각 가중치를 곱하고 더한다.

그럼 현재 은닉상태를 구하는 공식은 다음과 같다.

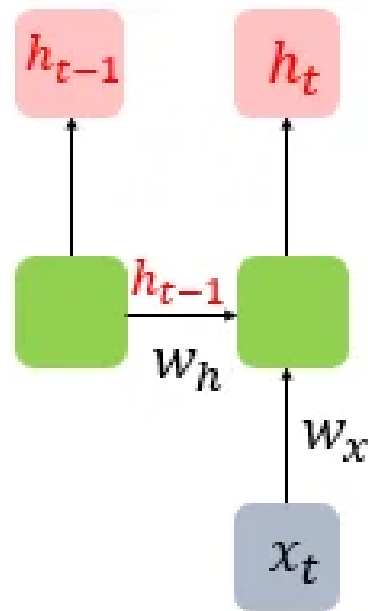
$$h_t = \tanh(W_{xh} \times x_t + W_{hh} \times h_{t-1})$$

$\tanh$ 은 활성화함수. RNN에서는 활성화함수로 $\tanh$ 를 많이 쓴다고 한다
만일 여기서 출력을 구하고 싶다면 $y_t = W_{hy} \times h_t$ 공식에 따라 구하면 된다

즉, RNN은 모든 입력을 최대한 잘 설명할 수 있는 가장 적합한 가중치 3개를 구하는 모델이다.



출력층을 포함한 그림



입력층과 은닉층만을 표현한 그림

오른쪽을 먼저 보는 것을 추천한다

그림을 보면 이해가 더 쉬울 것이다. 초록색 블록을 은닉상태를 만드는 **셀**이라고 생각하면 좋다.

왼쪽의 이전 셀이 h_{t-1} 을 주면 W_h 과 곱하고, 들어온 x_t 와 W_x 를 곱하여 더한다
이를 활성화 함수 $\tanh$ 를 통과시켜 h_t 를 만들고, 출력을 만들 때는 W_y 와 h_t 를 곱해준다

은닉 상태에 대해

은닉상태에 대해 좀 더 알아보자. 참고로 **은닉상태 = 은닉층 = 레이어**이다.

기본적으로 은닉상태는 **셀(Cell)**로 표현된다. 참고로 셀 = 유닛이다.

교재에서는 셀이 **과거의 정보를 담고있다**고 표현하는데, 좀 더 풀어서 설명하자면
각각의 셀이 1초 전의 정보, 2초전의 정보, 이렇게 담고 있는 것이 **아니다**

오히려 셀은 **특정한 맥락**의 정보를 숫자로 표현한 것에 가깝다.

이래 말하니깐 더 어려울 거 같은데, 예시를 들어보자.

입력값: ["나는", "눈물이", "흘렀다"]를 각각 셀이 1개인 레이어와 4개인 레이어로 학습을 시킨다고 생각해보자

셀이 1개인 레이어는 은닉상태가 **[0.3]** 이런 식으로 표현이 된다. 모델이 스스로 특징 1개만 찾아낼 수 있다.

즉, 특정한 하나의 숫자에 계속 새로운 입력값을 집어넣어 덮어씌우게 된다.(이런식으로 문장이 길어지면 처음에 들어온 입력값은 가중치를 잃어감)

셀이 4개인 레이어는 은닉상태가 **[0.4, 0.1, 0.6, 0.5]** 이런 식으로 표현이 된다. 이게 뭘 말하는 거냐궁? 정보를 여러 측면으로 분석했다고 생각하면 편하다.

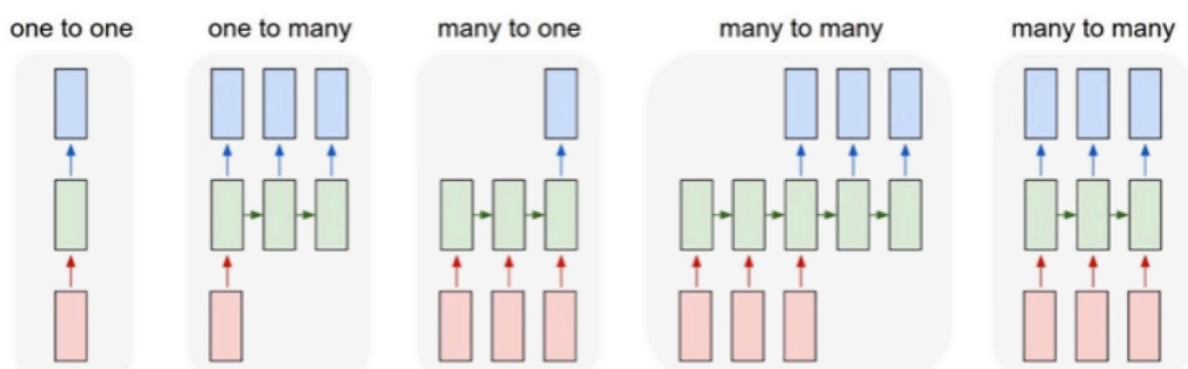
예를 들면 [기쁨, 놀라움, 슬픔, 우울함] 과 같이 특정 정보를 수치로 표현한 것이다. 문장의 길이가 길어져도 예전의 정보를 보존하고 있는 셀이 있을 거라 생각이 든다



Made with Napkin

셀의 크기 n 에 따라 W_{hh} 의 크기도 $n \times n$ 로 정해지게 된다

RNN 구조의 종류



일대일(One to One)

┃ 단일 입력과 단일 출력이 있는 구조

바닐라 신경망이라고도 한다고. Vanilla Neural Network

일대다(One to Many)

| 하나의 입력을 받아 여러 출력을 만드는 구조

좋은 예시로 **이미지 캡션을 생성하는**(이미지를 설명하는 문장을 만드는) RNN이 있다.

다대일(Many to One)

| 여러 입력을 받아 하나의 출력을 만드는 구조

여러 예시가 생각난다. 문장의 마지막 단어를 예측하는 모델, **문장의 감정을 분석하는 신경망** 등등등...

다대다(Many to Many)

| 많은 입력을 받아 여러 출력을 만드는 구조

가장 좋은 예시는 **번역 모델**! 문장에서 각각의 단어를 계속 번역된 단어로 출력하지 않는가.

순방향 패스 알고리즘

교재의 의사코드를 보고 이해하자. 어렵진 않음.

```
hidden_state = [0, 0, 0, 0]

sentence = ["I", "HATE", "recurrent", "neural"]

for word in sentence:
    hidden_state = np.tanh( np.dot(Wxh, word) + np.dot(Whh,
hidden_state) )
    prediction = f(np.dot(Wxh, word) + np.dot(Why, hidden_s
tate))
print(prediction)
```

케라스에서 RNN을 구현하기 위한 SimpleRNN을 알아보자

```
inputs = np.random.random([32, 10, 8])
```

```
simple_rnn = tf.keras.layers.SimpleRNN(4)
output = simple_rnn(inputs)
```

SimpleRNN은 **셀의 개수**를 인자로 받고, 인풋을 **[batch, timesteps, feature]**의 형태로 받는다.

- **batch**: 배치 사이즈.
- **timesteps**: 결과를 위해 모델이 과거를 얼마나 거슬러 올라가느냐
- **feature**: 특징의 개수

기본적으로 출력은 **[batch, units]**의 형태로 나온다.

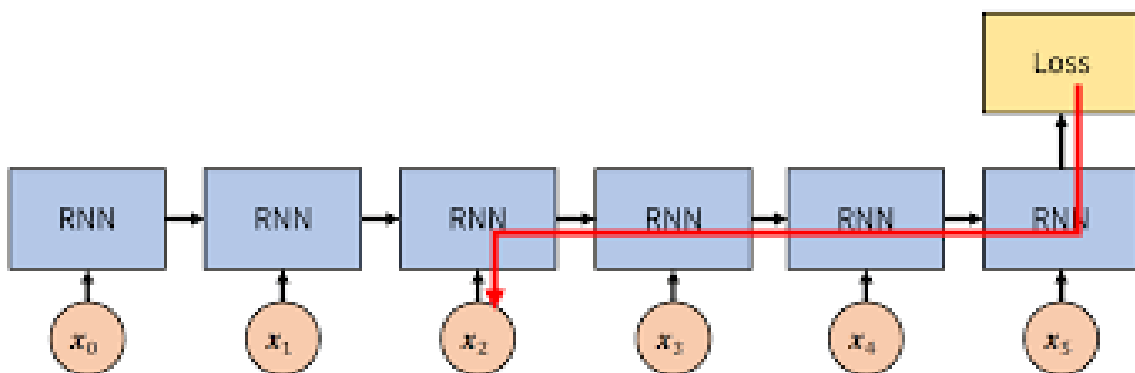
하지만 `return_sequences`를 True로 설정해주면 **[batch, timesteps, hidden_units]**의 형태로 준다

RNN의 학습

원래 FFNN에서는 계층을 통해 오류를 역전파하였습니다.

하지만? RNN에서는 시간을 거슬러 올라가면서 그래디언트를 역전파합니다.

이를 **BPTT(Backpropagation Through Time)**라고 합니다



그렇게... 이해하기 어렵진 않다.

BPTT에서도 마찬가지로 **그래디언트 폭발**, **그래디언트 소실**과 같은 문제들이 발생한다.

LSTM (Long Short Term-Memory)

이를 해결하기 위해 나온 모델이 LSTM입니다.

단순하게 말하면 정보를 가져갈 때, 버릴 때, 출력할 때 쓸 가중치를 각각 나누어 학습하자입니다

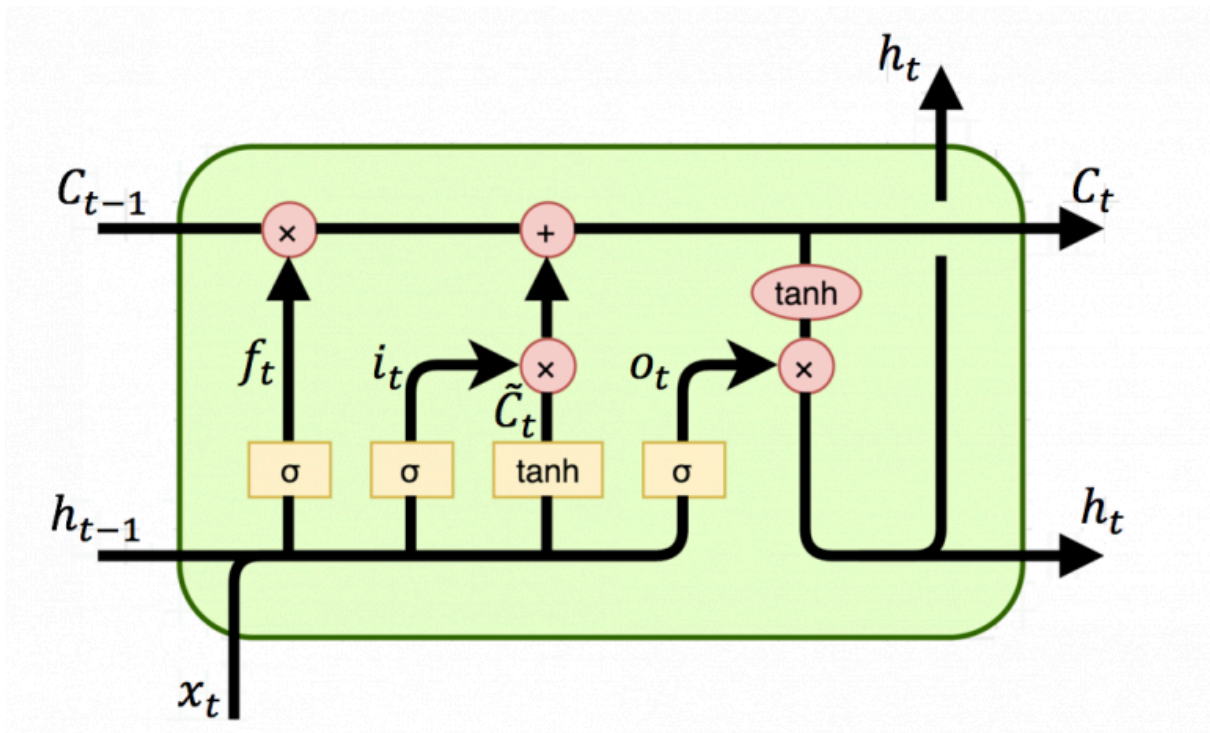


그림 너무 깔끔하고 좋아요...

처음 보면 복잡해 보일 수 있지만, 복잡한 것이 사실이다. 천천히 들여다보자.

LSTM에서 가장 중요한 것은 cell state인데, C_{t-1} 옆에 수평으로 연결된 선이 그 cell state이다.

이 cell state에 중요한 정보는 넣고, 필요없는 정보를 갖다 버리기 위해서 밸브가 3개가 있는 것이다

그 3개의 밸브는 다음과 같은 연산을 통해 0~1의 값으로 정해진다. 예상하시다시피 0에 가까울수록 버리는 경향이 크고, 1에 가까울수록 받아들이는 경향이 있다

- x_t (**현재 입력값**): 지금 막 들어온 따끈따끈한 정보 (예: 현재 단어, 오늘 주가).
- h_{t-1} (**이전 출력값**): 바로 전 단계에서 내보냈던 단기 기억.
- C_{t-1} (**이전 셀 상태**): 태초부터 지금까지 쭉쭉이 쌓여온 장기 기억 저장소(컨베이어 벨트).
- C_t, h_t (**결과물**): 위 재료들을 버무려서 새로 만들어낸 **현재의 장기 기억**과 **최종 출력값**.
- W_f (**망각 가중치**): 장기 기억에서 뭐 지울까 결정.
- W_i (**입력 가중치**): 현재 입력값을 얼마나 중요하게 저장할까를 결정.

- W_C (후보 가중치): 저장한다면 어떤 '내용'으로 바뀌서 저장할까? 결정. (입력값을 그대로 저장할수는 없으므로 특정한 공정을 거쳐 봐줘야함)
- W_o (출력 가중치): 업데이트된 장기 기억 중에서 어떻게 출력값을 정할것인지 결정.

모든 밸브는 다음과 같은 공식을 통해 정해집니다.

$$\text{Value} = \sigma(W \cdot [h_{t-1}, x_t] + b)$$

σ 는 시그모이드를 활성화 함수로 사용했다는 것을 알려줍니다. 값은 0 ~ 1의 형태로 나옴
저장할 내용은 다음과 같은 공식으로 정해집니다

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

그렇게 어렵진 않죠? 이제 이들을 합하여 cell state를 업데이트합니다

$$C_t = (f_t \otimes C_{t-1}) \oplus (i_t \otimes \tilde{C}_t)$$

대충 보면,,, 망각 밸브와 이전 셀 상태를 원소별 곱해주고, 입력 밸브와 현재 저장할 내용을 원소별 곱해줘서 서로 더해지면 됩니다.

최종 출력은 다음과 같은 식을 가집니다

$$h_t = o_t \otimes \tanh(C_t)$$

내가 보기엔 그림을 잘 못 그렸습니다. 그림을 보면 괜히 더 어려워 보이는데 있어요.

LSTM이 가중치 소실에 강한 이유

1. 덧셈 기반의 셀 업데이트

RNN은 무조건 곱셈을 했는데, LSTM은 덧셈 기반으로 셀을 업데이트 한다. 이 때문에 미분을 해도 그대로 가중치를 통과시키게 됨

2. 망각 게이트의 제어

기울기가 너무 커지거나, 너무 작아지는 문제를 망각 게이트가 적절하게 제어해준다.